Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 02 de noviembre de 2018

Padrón: **Apellido y Nombre:**

Punteros: APROBADO – DESAPROBADO **TDA:** APROBADO – DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){
  int *A, *C, *F;
  char *D, *E;
  char **B;
  char G;
  int H;

  H = 66 + ULTIMO_DIGITO_PADRON;
  G = 'A';
  A = new int;
  (*A) = H;
  C = &H;
  F = A;
  H++;
  cout << H << (*A) << (*C) << (*F) << endl;

  D = new char[4];
  E = D + 1;
  B = &D;
  D[0] = G;
  D[1] = 'C';
  cout << (*D) << (*E) << (**B) << endl;

  E[1] = *(D + 1);
  E[2] = G;
  E = &G;
  G = 'B';
  cout << (*B)[2] << (*E) << *(D + 2) << endl;

  E = D;
  D = (char*) F;
  (*A)--;
  cout << (**B) << (*A) << (*D) << endl;

  // liberar la memoria

  return 0;
}
```

2. Implementar para la clase Lista<T> con una estructura simplemente enlazada el siguiente método, indicando pre y post:

```
T remover(unsigned int posicionElemento);
```

3. Implementar el método `buscarMejorViaje` de la clase `AgenteDeViajes` a partir de las siguientes especificaciones:

```
class AgenteDeViajes {

  public:
    /* post: busca en 'viajesDisponibles' aquel Viaje de menor costo, que no tenga ninguna Escala en
     *       'lugaresYaVisitados' y cuya duración total sea menor o igual a 'duracionMaxima'.
     */
    Viaje* buscarMejorViaje(Lista<Viaje*>* viajesDisponibles,
                            Lista<string>* lugaresYaVisitados, unsigned int duracionMaxima);
}
```

```
class Viaje {

  public:

    /* post: Viaje creado sin Escalas por el costo indicado.
     */
    Viaje(unsigned int costo);

    /* post: devuelve el costo del viaje en $.
     */
    unsigned int obtenerCosto();

    /* post: devuelve todas las Escalas que lo conforman.
     */
    Lista<Escala*>* obtenerEscalas();

    /* post: libera todas las Escalas del Viaje.
     */
    ~Viaje();
};
```

```
class Escala {

  public:

    /* post: crea una Escala en el lugar indicado,
     *       por los días dados en 'duracion'
     */
    Escala(string lugar, unsigned int duracion)

    /* post: devuelve el nombre del lugar.
     */
    string obtenerLugar();

    /* post: devuelve la duración en días
     */
    unsigned int obtenerDuracion();

}
```

4. Diseñar la especificación e implementar el TDA BlancoDeArqueria con las siguientes operaciones:

- crearlo indicando como parámetro la cantidad máxima de tiros a recibir.
- recibirTiro: recibe un tiro, indicando como parámetro su puntaje [1..10].
- obtenerPuntuacionTotal: devuelve la puntuación total, acumulada durante todos los tiros recibidos.
- obtenerPuntuacionPromedio: devuelve el promedio de puntos por tiro.
- obtenerPuntuacionMejorTiro: devuelve la puntuación del mejor tiro recibido.



Ejemplifique el uso de la clase, indicando el resultado esperado.

Los alumnos que tienen aprobado el parcialito de punteros y/o TDA no deben realizar el ejercicio 1 y/o 4 respectivamente.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

```
***0***
67666766
ACA
CBC
A65A
***1***
68676867
ACA
CBC
B66B
***2***
69686968
ACA
CBC
C67C
***3***
70697069
ACA
CBC
D68D
***4***
71707170
ACA
CBC
E69E
***5***
72717271
ACA
CBC
F70F
***6***
73727372
ACA
CBC
G71G
***7***
74737473
ACA
CBC
H72H
***8***
75747574
ACA
CBC
I73I
***9***
76757675
ACA
CBC
J74J
***TODOS***
delete A
delete[] E
```

Los alumnos que tienen aprobado el parcialito de punteros y/o TDA no deben realizar el ejercicio 1 y/o 4 respectivamente.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas